Southern New Hampshire University

CS:300 Analysis and Design

Project One

Southern New Hampshire University

Joshua Evans

October 11, 2026

Southern New Hampshire University

ABCU Advising Program

Pseudocode and Runtime Analysis

This document contains the pseudocode and runtime analysis for the ABCU advising program. The program loads course data from a file, stores it in three different data structures, and allows the user to print course information and sorted course lists. The pseudocode includes the vector, hash table, and tree versions along with menu options for each one and the runtime analysis compares the performance of the three structures and ends with my recommendation based on the results.

// Vector Pseudocode //

// Load a file and build a vector //

START PROGRAM

CREATE Vector<Course> courseList

OPEN the course file

IF file does not open

PRINT “File error.”

STOP PROGRAM

END IF

FOR each line in file

SPLIT line by commas into tokens

IF tokens count < 2

```
PRINT “Line format error.”

CONTINUE

END IF


SET courseNumber = first token

SET courseTitle = second token

CREATE empty Vector prereqList

FOR each token after second

ADD token to prereqList

END FOR


CREATE Course tempCourse

tempCourse.number = courseNumber

tempCourse.title = courseTitle

tempCourse.prerequisites = prereqList

ADD tempCourse to courseList

END FOR

CLOSE file


// Print one course //

FUNCTION searchCourse(courseList, courseNumber)

FOR each course in courseList

IF course.number == courseNumber

PRINT course.number and course.title
```

```
IF course.prerequisites is empty

PRINT “No prerequisites.”

ELSE PRINT “Prerequisites: ”

FOR each prereq in course.prerequisites

PRINT prereq

END FOR

END IF

RETURN

END IF

END FOR

PRINT “Course not found.”

END FUNCTION


// Print sorted list //

FUNCTION printCourseListVector(courseList)

IF courseList is empty

PRINT “No courses.”

RETURN

END IF


CREATE sortedList = courseList

SORT sortedList by course.number

FOR each course in sortedList

PRINT course.number and course.title
```

```
END FOR

END FUNCTION


// Menu //

FUNCTION mainVector()

SET courseList empty

SET choice = 0

WHILE choice not 9

PRINT menu options

READ choice

IF choice = 1

ASK for file name

courseList = load file

ELSE IF choice = 2

printCourseListVector(courseList)

ELSE IF choice = 3

ASK for course number

searchCourse(courseList, courseNumber)

ELSE IF choice = 9

PRINT “Goodbye.”

ELSE

PRINT “Invalid option.”

END IF

END WHILE
```

END FUNCTION

// Hash Table Pseudocode //

// Load a file and build a hash table //

OPEN file

IF file fails

PRINT “Error.”

STOP

END IF

CREATE HashTable courseTable

CREATE list allCourseNumbers

FOR each line

SPLIT line

IF tokens < 2

PRINT “Format error.”

CONTINUE

END IF

ADD tokens[0] to allCourseNumbers

END FOR

RESET file

FOR each line

SPLIT line

SET courseNumber = tokens[0]

```
SET courseName = tokens[1]

CREATE prereqList

FOR each token after second

IF prereq not in allCourseNumbers

PRINT “Error.”

CONTINUE

END IF

ADD prereq to prereqList

END FOR

CREATE Course tempCourse

SET fields

INSERT tempCourse into courseTable

END FOR

// Print one course //

FUNCTION printCourse(courseTable, courseNumber)

COMPUTE key

SET node = bucket at key

WHILE node not null

IF node.course.number == courseNumber

PRINT course info

IF prereq list empty

PRINT “No prerequisites.”

ELSE

PRINT prereqs
```

```
FOR each prereq
PRINT prereq
END FOR
END IF
RETURN
END IF
node = node.next
END WHILE
PRINT “Course not found.”
END FUNCTION

// Print sorted list //
FUNCTION printAllCourses(courseTable)
CREATE empty list keys
FOR each bucket
FOR each node
ADD node.course.number to keys
END FOR
END FOR
SORT keys
FOR each key
PRINT courseTable[key].number and name
END FOR
END FUNCTION
```

// Menu //

```
FUNCTION mainHash()

SET courseTable empty

SET choice = 0

WHILE choice not 9

PRINT menu

READ choice

IF choice = 1

load file

ELSE IF choice = 2

printAllCourses(courseTable)

ELSE IF choice = 3

ASK for course number

printCourse(courseTable, courseNumber)

ELSE IF choice = 9

PRINT “Goodbye.”

ELSE

PRINT “Invalid option.”

END IF

END WHILE

END FUNCTION
```

// Tree Pseudocode //

// Load a file and build a tree //

OPEN file

IF fail

PRINT “Error.”

STOP

END IF

CREATE tree courseTree

CREATE list allCourseNumbers

FOR each line

SPLIT line

IF tokens < 2

PRINT “Format error.”

CONTINUE

END IF

ADD tokens[0] to allCourseNumbers

END FOR

RESET file

FOR each line

SPLIT line

SET courseNumber = tokens[0]

SET courseName = tokens[1]

CREATE prereqList

FOR each token after second

IF prereq not in allCourseNumbers

PRINT “Error.”

CONTINUE

END IF

ADD prereq

END FOR

CREATE Course tempCourse

INSERT tempCourse into tree

END FOR

// Print one Course //

FUNCTION printCourse(courseTree, courseNumber)

SET current = root

WHILE current not null

IF current.course.number == courseNumber

PRINT course info

IF prereq empty

PRINT "No prerequisites."

ELSE

PRINT prereqs

FOR each prereq

PRINT prereq

END FOR

END IF

RETURN

END IF

IF courseNumber < current.course.number

current = current.left

ELSE

current = current.right

END IF

END WHILE

PRINT “Course not found.”

END FUNCTION

// Print Sorted List //

FUNCTION printAllCourses(courseTree)

CALL inOrder(root)

END FUNCTION

FUNCTION inOrder(node)

IF node is null

RETURN

END IF

CALL inOrder(node.left)

PRINT node.course.number and node.course.name

IF prereq empty

PRINT “No prerequisites.”

ELSE

PRINT prereqs

FOR each prereq

PRINT prereq

END FOR

END IF

CALL inOrder(node.right)

END FUNCTION

// Menu //

FUNCTION mainTree()

SET root null

SET choice = 0

WHILE choice not 9

PRINT "Menu"

READ choice

IF choice = 1

load file

ELSE IF choice = 2

printAllCourses(root)

ELSE IF choice = 3

ASK for course number

printCourse(root, courseNumber)

ELSE IF choice = 9

PRINT "Goodbye"

ELSE

PRINT “Invalid option.”

END IF

END WHILE

END FUNCTION

Runtime Analysis

File reading and course creation are the same for all three structures.

Step	Cost	Times	Total
Read line	1	n	n
Split line	1	n	n
Check format	1	n	n
Create course object	1	n	n
Loops prereqs	1	n	n

Insert Cost:

Vector insert: 1 each → n total → O(n)

Hash table insert: 1 each average → n total → O(n)

Tree insert: log n each → n log n total → O(n log n)

Final Big O:

Vector load time: O(n)

Hash table load time: O(n)

Tree load time: O(n log n)

The Advantages and Disadvantages of Each One

Vector

PROS: Simple, easy to sort, and easy to print

CONS: Slow search and linear lookup

Hash Table

PROS: Fast search, fast insert, great for pulling one course

CONS: Sorting is still needed and buckets use memory.

TREE

PROS: Sorts output naturally and decent search

CONS: Harder to code and can get unbalanced

My Recommendation based on my findings

Personally, I would go with the hash table because it gives fast lookups. This is the main thing the advisors want when they pull up a single course for a student. Sorting the keys is very easy and is only done when printing the full list. Overall, it hits a good balance and feels smooth to use.